COD Risk Decision Engine | Coding Agent Build Prompt

Copy this specification into your coding agent together with orders_sample.csv and the original take-home assignment PDF. Implement the decision engine first. Do not silently change business rules. Flag unresolved assumptions clearly.

1. Project overview and business context

Build an explainable, configurable pre-shipment decision engine for a D2C haircare brand using Shopify. Approximately 3,000 monthly orders; 55% COD; COD RTO approximately 28%; prepaid RTO approximately 4%; average order value Rs 649; delivered gross margin approximately 62%. Each RTO costs Rs 65 forward shipping + Rs 65 return shipping + Rs 25 packaging/handling + 8% x Rs 180 expected damage = Rs 169.40. Marketing reports Rs 300+ acquisition cost per new Meta customer. Acquisition cost already spent is sunk and must not be counted again as incremental shipping cost.

The business objective is to reduce avoidable RTO loss WITHOUT rejecting genuine first-time buyers merely due to estimated risk. The engine accepts an incoming COD order and emits exactly one primary action: SHIP_AS_IS, SHIP_AFTER_CONFIRMATION, NUDGE_TO_PREPAID, or HOLD_CANCEL. Every action requires a concise, factual, plain-English operations explanation. This is Part 1 of a larger assignment; do not implement a full Shopify site, production WhatsApp integration, or the founder dashboard in this phase.

2. Inputs and dataset contract

Read orders_sample.csv. Actual columns: order_id, order_datetime, customer_name, phone, address, city, state, pincode, order_value, items, num_items, customer_prior_orders, customer_prior_rto, utm_source, courier, payment_method, final_status. Dataset contains 2,018 rows. Normalize case/whitespace in statuses, text, and sources; parse dates; treat pincode as a string to preserve leading zeros; check duplicates, missing fields, zero/invalid prices, impossible history (prior RTO > prior orders), and label validity. Log exclusions and never silently discard records.

Prior successful deliveries = customer_prior_orders - customer_prior_rto. Trust score = prior successful deliveries - customer_prior_rto = customer_prior_orders - 2 * customer_prior_rto. New customer means customer_prior_orders == 0, not a guess from name or phone. Do not use final_status or any future outcome as an inference-time input. Respect chronological evaluation to avoid leakage.

3. Agreed business decision rules (implement in precedence order)

Priority	Condition	Primary action
1	Input unusable / undeliverable address	HOLD_CANCEL with HOLD and request correction; do not falsely label as fraud
2	Repeat: successful deliveries = 0 AND trust score <= -2	HOLD_CANCEL with REJECT policy; configurable; include reason and audit log
3	Repeat: trust score >= 2	SHIP_AS_IS (unless hard address validation blocks shipment)
4	Repeat: trust score < 0, with >= 1 prior successful delivery	SHIP_AFTER_CONFIRMATION; WhatsApp mandatory
5	Repeat: trust score 0 or 1	Predict COD RTO probability; SHIP_AS_IS or SHIP_AFTER_CONFIRMATION via financial threshold

Priority	Condition	Primary action
6	New: predicted COD RTO probability > 40%	NUDGE_TO_PREPAID, 5% off capped at Rs 50; only new customers
7	New: risk <= 40%	SHIP_AS_IS or SHIP_AFTER_CONFIRMATION via financial threshold

Clarifications: Exactly -2 is rejected ONLY when prior successes are zero. Example 1 success / 3 RTOs (score -2) must NOT auto-reject; confirmation is required. Repeat customers NEVER receive prepaid nudges. First-time customers are NEVER automatically rejected solely because their model-predicted RTO probability is high. A completely unusable address is an operational HOLD for correction, not a risk-based new-customer rejection. For low-risk new customers, WhatsApp is not mandatory.

4. Confirmation and prepaid event contracts

The decision engine should return workflow metadata, not actually send messages. Confirmation: valid customer replies are YES and NO; after 24 hours without confirmation emit REMINDER_DUE; after 48 hours without confirmation emit HOLD_TIMEOUT; YES makes COD eligible to ship; NO cancels the COD order. For prepaid nudge, ACCEPT means initiate prepaid payment; ship only after verified payment success. DECLINE means fall back to mandatory COD confirmation (YES/NO) with a 48-hour window. Distinguish prepaid-offer DECLINE from COD-confirmation NO. Ambiguous/random messages do not count as YES. Implement pure state-transition functions or interfaces for future mock simulator; no real WhatsApp account needed.

Do not assume a customer who clicks YES will definitely accept delivery. Preserve timestamps, message states, retries/idempotency, and transition reasons. Make the 48-hour clock policy explicit (e.g., from first confirmation request); do not reset it accidentally on reminders. The later WhatsApp mock will consume these transitions.

5. Risk prediction: baseline and validation

Start with a transparent, reproducible baseline (logistic regression plus a simple rules/segment baseline). Train on earlier orders and evaluate on later orders; include a naive overall-rate benchmark. Candidate features: utm_source (meaningful influence), order_value (risk prediction AND separately financial economics), courier, simple address completeness indicators, and city. City influence must be deliberately small and city ALONE must never reject an order. Pincode history is sparse; use shrinkage / minimum-sample fallback instead of raw tiny-group rates. Never hardcode unsupported feature percentages as calibrated probabilities. Compare models by calibration/Brier score, ROC-AUC, segment metrics and actual financial backtest; choose the simplest defensible model.

Address evaluation: first deterministic validation for missing/invalid critical delivery fields; optionally call an LLM ONLY for ambiguous/Hinglish address semantics, placeholder names, or suspicious free text. Categories: VALID (illustrative quality score +1), INCOMPLETE (+0.5), SUSPICIOUS (0; raises risk modestly), BAD_UNDELIVERABLE (HOLD for correction). These scores are NOT probabilities; calibrate or explicitly configure their influence. Do not let an LLM alone label a person fraudulent or auto-cancel an order. Log LLM output, rationale, errors and fallback; minimize PII sent to third parties. Use a mock/fallback mode without API keys.

6. Financial decision threshold and economics

Delivered contribution estimate M = 0.62 * order_value. RTO direct expected loss L = Rs 169.40 under assignment assumptions. Expected COD contribution = (1-p)*M - p*L. For WhatsApp, let a = fraction of potential RTOs prevented by mandatory confirmation, b = fraction of otherwise delivered orders lost because confirmation blocks them, and C = message cost. Incremental confirmation value = p*a*L - (1-p)*b*M - C. Confirm if incremental value > 0. Break-even risk threshold p* = (b*M + C)/(a*L + b*M), if denominator positive. These are scenario assumptions, NOT learned from orders_sample.csv; make a,b,C configurable and surface them in output.

Default illustrative scenario ONLY: a=0.50, b=0.03, C=Rs 1, gross margin=0.62. For order Rs 649, break-even threshold approximately 13.5%. For new customers, apply prepaid nudge first when p > 0.40, otherwise compare p to the dynamic confirmation threshold. For repeat trust score 0/1, use dynamic confirmation threshold. Show expected COD contribution, expected incremental confirmation contribution, assumptions, and reason in every applicable decision. Do not represent historical outcome labels as causal evidence of WhatsApp effectiveness.

7. Prepaid nudge economics (new customers only)

Default offer: min(5% of order_value, Rs 50). If p > 0.40 for a NEW customer, emit NUDGE_TO_PREPAID. The assignment gives overall prepaid RTO 4%; it does not establish the rate for converted high-risk COD buyers. Make assumed converted-prepaid RTO probability configurable, initially 0.04 as a scenario. If discount D is applied only to delivered orders, expected prepaid contribution = (1-q)*(M-D) - q*L; explicitly state this assumption. If discount/payment fees are incurred on a different basis, implement that accounting correctly. Expose a sensitivity analysis over prepaid acceptance rates and confirmation effectiveness; declined prepaid offers fall back to COD confirmation, not cancellation. Do not claim nudge profitability without a response-rate assumption.

8. Configuration, output and API requirements

All thresholds must be editable without code changes, via a human-readable YAML/JSON config and optionally a simple settings endpoint. Include trust thresholds, 40% nudge cutoff, 5% discount, Rs 50 cap, 24h reminder, 48h deadline, a,b,C, q, city/pincode smoothing, model version, and LLM mode. Validate config ranges and fail clearly on invalid values. Keep rules deterministic, ordered and unit-tested.

Provide a FastAPI endpoint POST /decide accepting an order payload matching the CSV input fields (without final_status), GET /config for readable settings, and optional POST /simulate-event for pure workflow transitions. Return: order_id; primary_action; sub_action (e.g., HOLD_ADDRESS, REJECT_REPEAT, CONFIRM_COD); customer_type; prior_successes; trust_score; predicted_rto_probability and model version; financial estimates with assumptions; prepaid offer if relevant; required next step; short human-readable reason; triggered rule IDs; warnings; audit timestamp. Return structured validation errors. Include sample requests and responses in README.

9. Backtesting and evaluation: no fabricated results

Create an offline backtest using the historical CSV and a chronological holdout. Report exact counts in all four primary buckets, observed historical RTO rate by bucket, and confusion/calibration metrics where appropriate. Use historical outcomes only for counterfactual scenario calculations; the CSV has NO WhatsApp response, acceptance or conversion observations. Evaluate multiple scenarios for confirmation efficacy, genuine-order loss and prepaid acceptance. Report gross avoided RTO cost, lost genuine delivered contribution, messaging/discount costs, and NET incremental contribution versus shipping every COD order. Explicitly label assumptions. Repeat the test after the final corrected reject rule; do not reuse previously stated conversation numbers without independently reproducing them.

Guard against leakage from duplicated IDs and repeated customer history; explain what can and cannot be inferred causally. Show how often rejected historical orders were actually delivered. Never optimize only for lower RTO rate at the expense of contribution or new-customer retention.

10. Implementation boundaries and deliverables

Preferred lightweight stack: Python 3.12+, FastAPI, pandas, scikit-learn, Pydantic, pytest; SQLite optional for audit logs; config YAML/JSON. Build a small maintainable repo: data validation/cleaning, model training/evaluation, decision policy, economics, address checks, workflow transitions, API, config, tests, and README. Avoid unnecessary infrastructure. Do not build a production e-commerce storefront, production WhatsApp integration, or polished dashboard yet. Design stable interfaces for these later components.

Tests must cover: new vs repeat, every trust-score boundary (including -2 with and without successful orders), p=0.40 boundary, dynamic threshold boundary, zero/missing/invalid prices, invalid histories, missing addresses, city-only high risk, prepaid accept/decline, YES/NO/random reply, 24h reminder, 48h timeout, repeated callbacks, LLM failure, and

config edits. Include reproducible fixtures and seeded experiments.

Deliver: working code, requirements/lockfile, example environment file, sample config, commands to train and run, documented API examples, unit tests, backtest report, short decision log, and AI usage log template. Repo must run locally in under 10 minutes without paid API credentials using fallback mode. Explain where an LLM helps and why simple arithmetic/rules should not use it.

11. Agent execution instructions

FIRST: inspect both attached files and summarize schema, quality problems, and implementation plan; identify contradictions or missing business inputs. SECOND: implement only the decision engine and supporting backtest/tests. THIRD: run tests and provide real outputs with exact commands. FOURTH: list assumptions, known limitations, and questions before expanding to WhatsApp UI or dashboard. Never silently substitute arbitrary numerical weights, invent response data, or claim that a simulated scenario is a measured business result. Favor correct, auditable decisions over complexity.

Source of truth: original take-home PDF for required deliverables and financial inputs; this specification for our jointly chosen policy. If conflicts arise, surface them explicitly instead of hiding them.